phone_app/lib/phone_app/conversations/query/sms_message_store.ex
```
def load_messages_with(contact) do
  from(
    m in SmsMessage,
    where: m.contact_id == ^contact.id,
    order_by: [desc: m.inserted_at],
    preload: [:contact]
  )
  |> Repo.all()
end
```

This sure does feel a lot like SQL! The close connection to SQL makes it easy to convert handwritten queries into an application.

Our code works because of the `import Ecto.Query` function at the top of this module. Typically, you'll use this import anytime your code calls the `Ecto.Query.from/2` function.

Let's break apart this query. The code `m in SmsMessage` tells the query builder that we are assigning the schema `SmsMessage` to the variable `m`. We use this to reference columns on the table. Then, `where: m.contact_id == ^contact.id` filters the query based on the `contact_id` column equaling the value of `contact.id`.

The caret symbol (^) is used to insert the contact's id value into the query. Don't confuse this with the pin operator (^) that we used in pattern matching. It's just the symbol that Ecto's authors chose to use.

The `order_by: [desc: m.inserted_at]` line sorts our query by the `inserted_at` column. Finally, `preload: [:contact]` tells Ecto that we want it to load the `contact` association. This works similarly to ActiveRecord, where an additional query is made to load all of the contact associations.

Also like ActiveRecord, we have the choice to join the contact association into our query and preload it without any additional queries. You don't need to type out the following code, but here's what it would look like to join in the contact and preload using the join data:

phone_app/lib/phone_app/conversations/query/sms_message_store.ex
```
def example_load_messages_with_join(contact) do
  from(
    m in SmsMessage,
    join: c in assoc(m, :contact),
    where: m.contact_id == ^contact.id,
    order_by: [desc: m.inserted_at],
    preload: [contact: c]
  )
  |> Repo.all()
end
```